Multi-Region AWS VPC Peering

with EC2 Instances, Apache Web Server & Terraform

Complete Project Documentation — Step-by-Step Guide + Network Flow + Diagrams

Author	Ritik
Primary Region	us-east-1 | VPC CIDR: 10.0.0.0/16
Secondary Region	us-west-2 | VPC CIDR: 10.1.0.0/16
Instance Type	t3.micro — Amazon Linux 2023
Web Server	Apache HTTPD — installed via user_data on boot
Connectivity	Cross-Region VPC Peering (private, no public internet)
IaC Tool	Terraform | AWS Provider ~> 6.43.0
Date	22-05-2026

---

1. Project Overview

This project uses Terraform to provision a complete dual-region AWS infrastructure. Two EC2 instances — one in us-east-1 (Primary) and one in us-west-2 (Secondary) — are each deployed inside their own VPC and connected via a cross-region VPC Peering connection, enabling fully private communication between the two regions without traversing the public internet.

- Dual-region Terraform deployment using provider aliases (aws.primary / aws.secondary)
- Two isolated VPCs with non-overlapping CIDRs: 10.0.0.0/16 and 10.1.0.0/16
- Cross-region VPC Peering with bidirectional routing — instances ping each other privately
- Dynamic data sources: AZs and latest Amazon Linux 2023 AMI fetched at plan time
- SSH key pairs generated per region and registered via AWS EC2 key-pair import
- Apache HTTPD auto-installed on boot via user_data shell scripts
- Security groups: SSH (22), HTTP (80), ICMP — plus full egress
- Outputs: public IPs of both instances printed after terraform apply

---

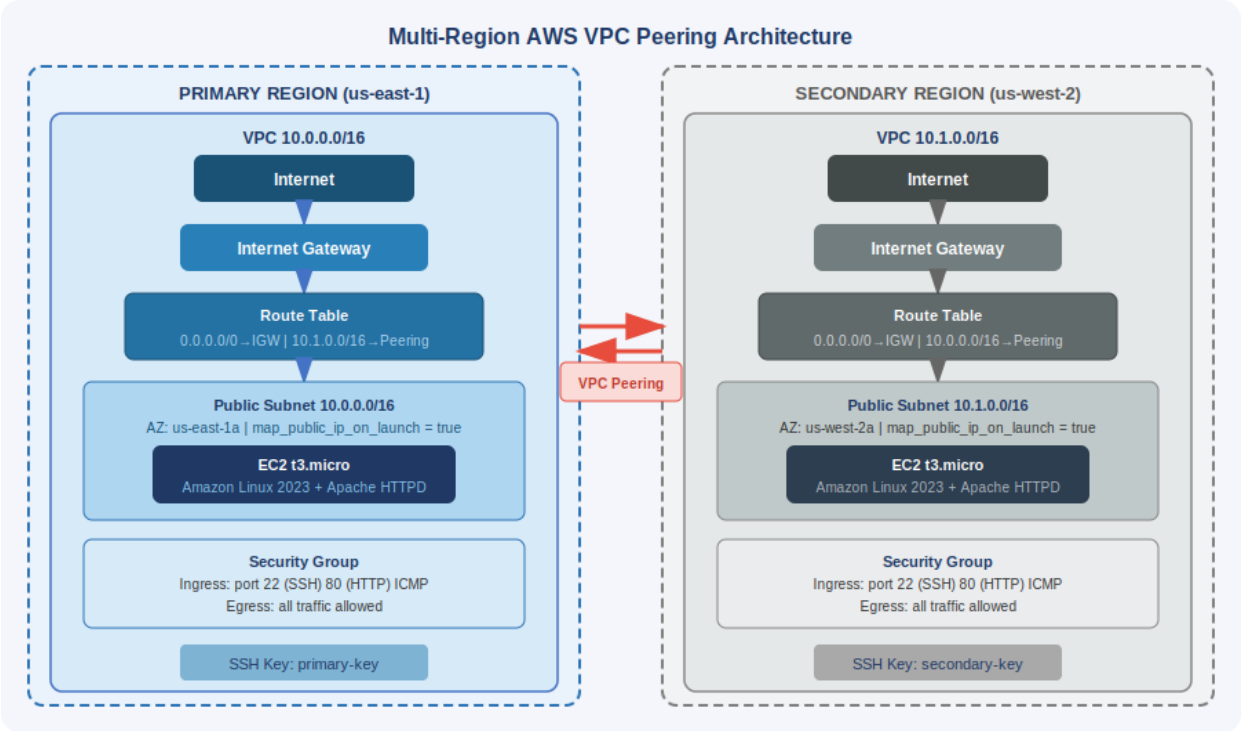
2. Project File Structure

Project File Structure

```
multi-region-vpc-peering/
├── providers.tf      ← dual AWS provider aliases (primary + secondary)
├── variables.tf      ← regions, CIDRs, instance type
├── data.tf           ← dynamic AZs + latest AL2023 AMI per region
├── main.tf           ← all infra: VPC, IGW, RT, SG, EC2, peering
├── outputs.tf        ← public IPs of both instances
├── backend.tf        ← (commented) S3 remote state config
├── primary.sh        ← user_data: yum install httpd + index.html
├── secondary.sh      ← user_data: yum install httpd + index.html
├── primary-key / primary-key.pub          ← SSH key pair (us-east-1)
├── secondary-key / secondary-key.pub      ← SSH key pair (us-west-2)
└── .gitignore · .terraform.lock.hcl · terraform.tfstate · terraform.tfstate.backup
```

3. Architecture Diagram

The diagram below shows the complete infrastructure: both VPCs, their subnets, Internet Gateways, Route Tables, Security Groups, EC2 instances, and the bidirectional VPC Peering connection.



4. Network Flow — What Each Resource Does & Why

Every resource serves a specific role in the network. This table explains the purpose and justification for each resource created in this project.

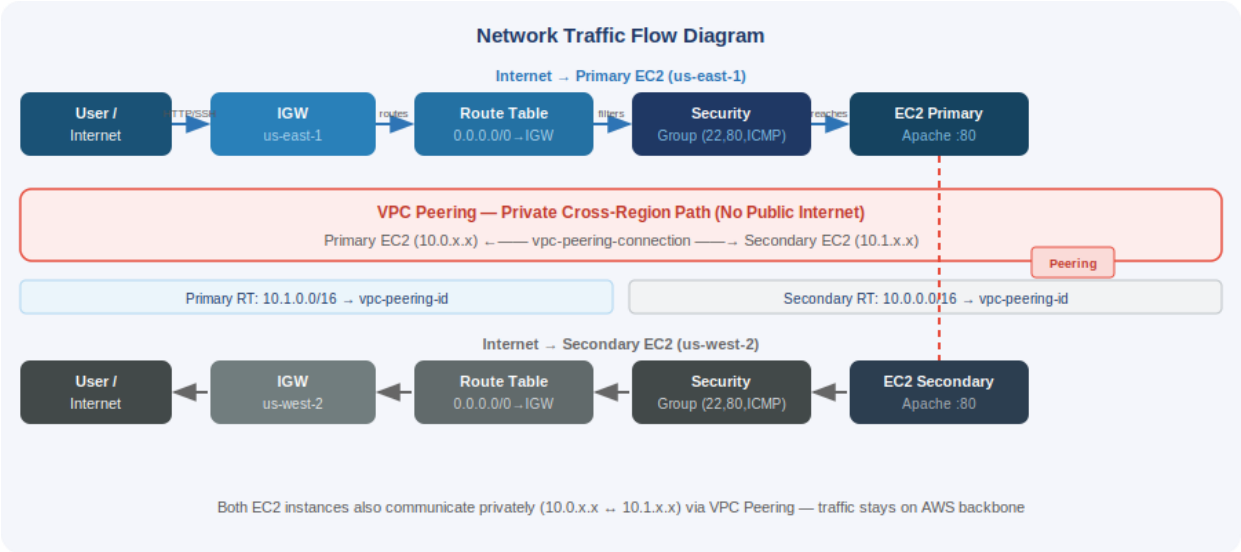
Resource	What It Does	Why It Is Needed
`aws_vpc (×2)`	Creates an isolated virtual network per region	Without a VPC you cannot launch EC2 or control networking. Non-overlapping CIDRs (10.0/16 & 10.1/16) are required for VPC peering routes to be unambiguous.
`enable_dns_hostnames / dns_support`	Lets VPC resolve DNS hostnames for instances	Needed so EC2 instances can resolve AWS service endpoints and each other by hostname inside the VPC.
`aws_subnet (×2)`	Carves a slice of the VPC CIDR for instances	Instances live in subnets, not directly in VPCs. map_public_ip_on_launch = true gives each EC2 a public IP automatically for SSH/HTTP access.
`data.aws_availability_zones (×2)`	Fetches available AZs at plan time per region	Avoids hardcoding AZ names. Uses names[0] dynamically so the config works regardless of which AZs exist in a region.
`data.aws_ami (×2)`	Fetches latest Amazon Linux 2023 AMI per region	AMI IDs differ per region and change with updates. A data source always resolves the newest, most-patched AMI automatically.
`aws_internet_gateway (×2)`	Provides a route in/out to the public internet	Without an IGW, public-subnet instances cannot send or receive internet traffic — SSH, HTTP, and yum installs during boot would all fail.
`aws_route_table (×2)`	Defines where traffic goes based on destination IP	Default VPC route tables have no internet route. Custom tables add 0.0.0.0/0 → IGW (internet) and 10.x.0.0/16 → peering (private cross-region).
`aws_route_table_association (×2)`	Binds the custom route table to the subnet	Without the association the subnet uses the default route table, which has no internet or peering routes — instances would be unreachable.
`aws_vpc_peering_connection`	Sends a peering request from primary to secondary	Opens the private cross-region channel between the two VPCs. auto_accept = false because cross-region acceptance must happen in the peer region.
`aws_vpc_peering_connection_accepter`	Accepts the peering request in us-west-2	Cross-region peering requires explicit acceptance in the peer region. The secondary provider accepts automatically via auto_accept = true.
`aws_route (primary→secondary)`	Adds route: 10.1.0.0/16 → peering connection	Without this, packets from the primary EC2 destined for 10.1.x.x have no path and are dropped. depends_on ensures peering is active first.
`aws_route (secondary→primary)`	Adds route: 10.0.0.0/16 → peering	Return path — peering must be bidirectional in both route tables. Without it, requests

	connection	reach secondary but replies cannot return.
`aws_security_group (×2)`	Stateful firewall controlling traffic per instance	Controls inbound/outbound traffic. Port 22 = SSH access. Port 80 = Apache web server. ICMP = ping to verify peering. Egress all = no restrictions outbound.
`primary/secondary-key pair`	Registers SSH public key with AWS per region	AWS key pairs are regional — each region needs its own. Allows password-free SSH login to EC2 instances using the private key file.
`aws_instance (×2)`	Launches the EC2 virtual machine	The actual compute resource. user_data runs the shell script on first boot to install Apache. key_name attaches the SSH key for secure access.
`primary.sh / secondary.sh`	Shell scripts run as user_data on EC2 first boot	Automates: yum update, yum install httpd, systemctl enable/start httpd, and writes a unique index.html — no manual SSH setup needed.

---

5. Network Traffic Flow Diagram

Shows how internet traffic reaches each EC2 instance, and how both EC2 instances communicate privately via VPC Peering.



5.1 Internet → EC2 Flow (per region)

- User sends HTTP (port 80) or SSH (port 22) to the instance public IP
- Traffic enters the region's Internet Gateway
- Route Table matches 0.0.0.0/0 → IGW — packet reaches the subnet

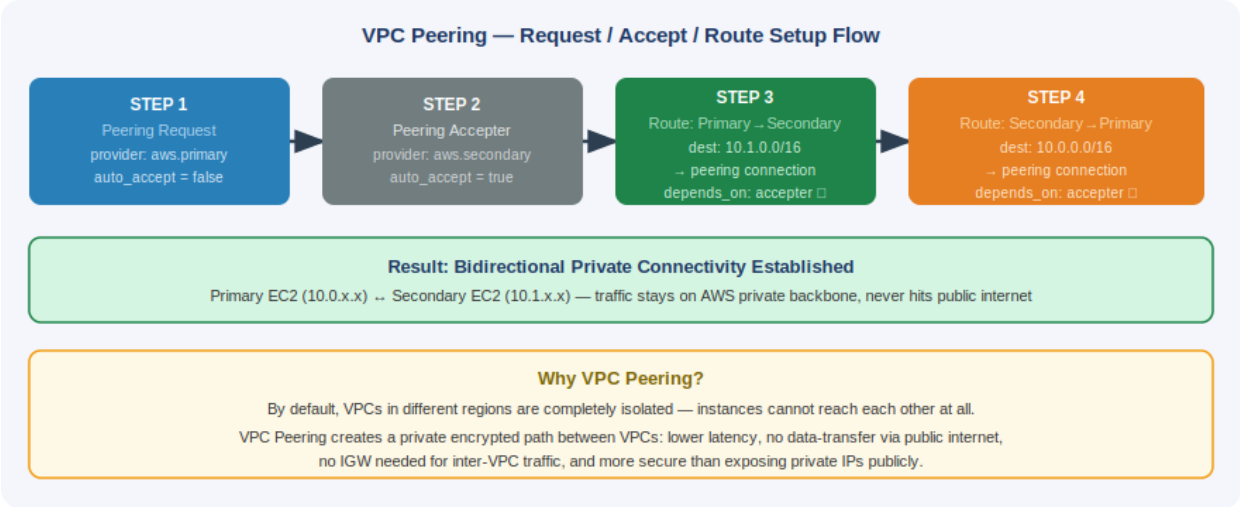
- Security Group evaluates ingress rules — port 22 and 80 are allowed from 0.0.0.0/0
- EC2 receives the request — Apache serves HTTP response, or SSH shell opens

5.2 EC2 Primary ↔ EC2 Secondary (VPC Peering path)

- Primary EC2 (10.0.x.x) sends a packet destined for 10.1.x.x
- Primary Route Table matches 10.1.0.0/16 → vpc-peering-connection-id
- AWS routes the packet over the private backbone — never touches public internet
- Secondary Route Table has 10.0.0.0/16 → vpc-peering-connection-id for the return path
- Secondary Security Group allows ICMP (ping), so connectivity can be verified
- Both instances can ping and curl each other using private IPs

---

6. VPC Peering — Request / Accept / Route Flow



---

7. Step-by-Step Implementation Procedure

Step 1 — Set Up Provider Aliases (providers.tf)

Two AWS provider blocks were created with aliases so Terraform can manage two regions from one config. Every resource specifies provider = aws.primary or provider = aws.secondary to deploy into the correct region.

```
provider "aws" { region = var.primary_region   alias = "primary"   }
provider "aws" { region = var.secondary_region  alias = "secondary" }
```

Step 2 — Generate & Register SSH Key Pairs

Two key pairs were generated locally (one per region) using ssh-keygen, then imported into AWS using the CLI. AWS key pairs are regional resources, so each region needs its own.

```
ssh-keygen -t rsa -b 2048 -f primary-key -N ""
ssh-keygen -t rsa -b 2048 -f secondary-key -N ""
aws ec2 import-key-pair --key-name primary-key   --public-key-material fileb://primary-key.pub   --region us-east-1
aws ec2 import-key-pair --key-name secondary-key --public-key-material fileb://secondary-key.pub --region us-west-2
```

Step 3 — Define Variables (variables.tf)

Input variables defined for regions, VPC CIDRs, and instance type. CIDRs are non-overlapping (10.0.0.0/16 and 10.1.0.0/16) — required for VPC peering routes to work correctly.

```
variable "primary_region"     { default = "us-east-1" }
variable "secondary_region"   { default = "us-west-2" }
variable "primary_vpc_cidr"   { default = "10.0.0.0/16" }
variable "secondary_vpc_cidr" { default = "10.1.0.0/16" }
variable "instance_type"      { default = "t3.micro" }
```

Step 4 — Create Data Sources (data.tf)

Data sources dynamically fetch: (1) available AZs per region — so AZ names are never hardcoded, and (2) the latest Amazon Linux 2023 AMI per region using a name-pattern filter, ensuring the most up-to-date, patched image is always used.

```
data "aws_availability_zones" "primary_az"  { provider = aws.primary   state = "available" }
data "aws_availability_zones" "secondary_az"{ provider = aws.secondary  state = "available" }
data "aws_ami" "primary_ami" {
  provider = aws.primary   most_recent = true   owners = ["amazon"]
  filter { name = "name"   values = ["al2023-ami-*-x86_64"] }
}
```

Step 5 — Create VPCs with DNS Support

Two VPCs provisioned — one per region — with enable_dns_support and enable_dns_hostnames = true so AWS assigns DNS names to instances inside the VPCs, which is needed for service discovery and AWS endpoint resolution.

```
resource "aws_vpc" "primary_vpc" {
  provider             = aws.primary
  cidr_block           = var.primary_vpc_cidr   # 10.0.0.0/16
  enable_dns_support   = true
  enable_dns_hostnames = true
}
```

Step 6 — Create Public Subnets

One public subnet per VPC. availability_zone is set from the data source (names[0]) to avoid hardcoding. map_public_ip_on_launch = true means every EC2 launched here automatically gets a public IP — no need to allocate/associate an Elastic IP separately.

```
resource "aws_subnet" "primary_subnet" {
  provider                = aws.primary
  vpc_id                  = aws_vpc.primary_vpc.id
  cidr_block              = var.primary_vpc_cidr
  availability_zone       = data.aws_availability_zones.primary_az.names[0]
  map_public_ip_on_launch = true
}
```

Step 7 — Create Internet Gateways

One IGW per VPC, attached to the VPC. Without an IGW, instances in public subnets have no path to/from the internet — SSH and HTTP would be impossible, and the user_data script's yum commands would fail on boot.

```
resource "aws_internet_gateway" "primary_igw" {
  provider = aws.primary
  vpc_id   = aws_vpc.primary_vpc.id
}
```

Step 8 — Create Route Tables with Internet Route

Custom route tables with a default 0.0.0.0/0 → IGW route. The peering routes (for private inter-region traffic) are added separately via aws_route resources after peering is established and accepted.

```
resource "aws_route_table" "primary_route_table" {
  provider = aws.primary
  vpc_id   = aws_vpc.primary_vpc.id
  route { cidr_block = "0.0.0.0/0"   gateway_id = aws_internet_gateway.primary_igw.id }
}
```

Step 9 — Associate Route Tables to Subnets

Route table associations explicitly link the custom route table to each subnet. Without this, subnets fall back to the default VPC route table which has no internet or peering routes — instances would be isolated.

```
resource "aws_route_table_association" "primary_rta" {
  provider       = aws.primary
  subnet_id      = aws_subnet.primary_subnet.id
  route_table_id = aws_route_table.primary_route_table.id
}
```

Step 10 — Create VPC Peering Connection (Request)

Peering request initiated from the primary provider, pointing at the secondary VPC in the secondary region. auto_accept = false because cross-region peering cannot self-accept — a separate accepter resource in the peer region is required.

```
resource "aws_vpc_peering_connection" "vpc_peering_request" {
  provider    = aws.primary
  vpc_id      = aws_vpc.primary_vpc.id
  peer_vpc_id = aws_vpc.secondary_vpc.id
  peer_region = var.secondary_region
  auto_accept = false
}
```

Step 11 — Accept the Peering Connection

The secondary provider explicitly accepts the pending peering request. auto_accept = true lets Terraform handle acceptance automatically. This resource must exist before the peering routes can be created — hence depends_on in the route resources.

```
resource "aws_vpc_peering_connection_accepter" "secondary_acceptor" {
  provider                  = aws.secondary
  vpc_peering_connection_id = aws_vpc_peering_connection.vpc_peering_request.id
  auto_accept               = true
}
```

Step 12 — Add Bidirectional Peering Routes

Two aws_route resources — one in each route table — create the bidirectional private path. depends_on = [accepter] ensures peering is fully accepted before routes are added; without this, Terraform may try to insert routes over a pending (not yet active) connection.

```
# Primary → Secondary
resource "aws_route" "primary_to_secondary" {
  provider                  = aws.primary
  route_table_id            = aws_route_table.primary_route_table.id
  destination_cidr_block    = var.secondary_vpc_cidr    # 10.1.0.0/16
  vpc_peering_connection_id = aws_vpc_peering_connection.vpc_peering_request.id
  depends_on                = [aws_vpc_peering_connection_accepter.secondary_acceptor]
}
# Secondary → Primary (mirror with aws.secondary provider, dest = primary CIDR)
```

Step 13 — Create Security Groups (port 22, 80, ICMP)

One security group per VPC. ICMP is especially important here — it allows using ping to verify that the VPC peering connection is working (primary EC2 pings secondary's private IP and vice versa). All egress is open.

```
ingress { from_port=22   to_port=22   protocol="tcp"   cidr_blocks=["0.0.0.0/0"] }
ingress { from_port=80   to_port=80   protocol="tcp"   cidr_blocks=["0.0.0.0/0"] }
ingress { from_port=-1   to_port=-1   protocol="icmp"  cidr_blocks=["0.0.0.0/0"] }
egress  { from_port=0    to_port=0    protocol="-1"    cidr_blocks=["0.0.0.0/0"] }
```

Step 14 — Write User Data Scripts

Shell scripts (primary.sh / secondary.sh) run automatically on first EC2 boot. They update packages, install Apache, enable it to auto-start on reboot, and write a unique HTML page identifying the region. Different page titles make it easy to verify which instance is serving.

```
#!/bin/bash
yum update -y && yum install -y httpd
systemctl enable httpd && systemctl start httpd
echo "<h1>Primary Region Apache Server</h1>" > /var/www/html/index.html
```

Step 15 — Launch EC2 Instances

Two EC2 instances launched using the dynamically fetched AMI, the correct subnet, security group, SSH key name, and the user_data shell script. Terraform resolves the correct deployment order via implicit resource dependencies.

```
resource "aws_instance" "primary_instance" {
  provider               = aws.primary
  ami                    = data.aws_ami.primary_ami.id
  instance_type          = var.instance_type
  subnet_id              = aws_subnet.primary_subnet.id
  vpc_security_group_ids = [aws_security_group.primary_sg.id]
  key_name               = "primary-key"
  user_data              = file("./primary.sh")
}
```

Step 16 — Define Outputs

Output values print both public IPs after terraform apply for immediate SSH/browser access without opening the AWS Console.

```
output "primary_public_ip"   { value = aws_instance.primary_instance.public_ip }
output "secondary_public_ip" { value = aws_instance.secondary_instance.public_ip }
```

Step 17 — Deploy with Terraform

```
terraform init      # Download AWS provider plugin
terraform validate  # Check for syntax errors
terraform plan      # Preview all resources to be created
terraform apply     # Deploy everything
```

Terraform provisions all 20+ resources in dependency order. Public IPs are printed in the output.

Step 18 — Verify the Deployment

- Open http://<primary_public_ip> in browser → sees 'Primary Region Apache Server'
- Open http://<secondary_public_ip> in browser → sees 'Secondary Region Apache Server'
- SSH into primary EC2 and ping secondary's private IP to confirm VPC peering works:

```
ssh -i primary-key ec2-user@<primary_public_ip>
ping 10.1.x.x     # Should get ICMP replies from the secondary EC2
```

8. CIDR & Routing Reference

Network / Route	Details
Primary VPC CIDR	10.0.0.0/16
Secondary VPC CIDR	10.1.0.0/16
Primary Subnet CIDR	10.0.0.0/16 (fills VPC, AZ: us-east-1a)
Secondary Subnet CIDR	10.1.0.0/16 (fills VPC, AZ: us-west-2a)
Primary RT — Route 1	0.0.0.0/0 → Primary Internet Gateway
Primary RT — Route 2	10.1.0.0/16 → vpc-peering-connection (private)
Secondary RT — Route 1	0.0.0.0/0 → Secondary Internet Gateway
Secondary RT — Route 2	10.0.0.0/16 → vpc-peering-connection (private)

---

9. Terraform Commands Reference

Download providers:

```
terraform init
```

Check syntax:

```
terraform validate
```

Preview changes:

```
terraform plan
```

Deploy:

```
terraform apply
```

Destroy all resources:

```
terraform destroy
```

Show state:

```
terraform show
```

---

10. Project Summary — Step by Step

Everything done in this project — concise summary

1. Configured two AWS provider aliases in providers.tf — aws.primary (us-east-1) and aws.secondary (us-west-2) — enabling single-config, dual-region Terraform deployments.
2. Generated two SSH key pairs (primary-key, secondary-key) locally and imported each into the correct AWS region using aws ec2 import-key-pair.
3. Defined variables for regions, non-overlapping VPC CIDRs (10.0.0.0/16 and 10.1.0.0/16), and instance type (t3.micro).

4. Created data sources to dynamically fetch available AZs and the latest Amazon Linux 2023 AMI per region — no hardcoded AZ names or AMI IDs.
5. Provisioned two VPCs (one per region) with DNS support and hostname resolution enabled — each with a unique CIDR block.
6. Created two public subnets with map_public_ip_on_launch = true so EC2 instances receive public IPs automatically, using a dynamically fetched AZ.
7. Attached one Internet Gateway to each VPC, enabling internet traffic in and out of the public subnets.
8. Created custom route tables with a default internet route (0.0.0.0/0 → IGW) and associated them to their respective subnets.
9. Initiated a cross-region VPC Peering Connection request from primary (auto_accept = false) and accepted it in the secondary region using aws_vpc_peering_connection_accepter.
10. Added two bidirectional peering routes — 10.1.0.0/16 → peering in primary RT, and 10.0.0.0/16 → peering in secondary RT — both with depends_on the accepter.
11. Created security groups in both VPCs allowing SSH (port 22), HTTP (port 80), and ICMP (ping) ingress, with all egress traffic allowed.
12. Wrote user_data shell scripts (primary.sh, secondary.sh) that auto-install and start Apache HTTPD on EC2 first boot with a unique index.html per region.
13. Launched two t3.micro EC2 instances — one per region — each connected to the correct subnet, security group, SSH key, and user_data script.
14. Defined outputs for both public IPs so they display immediately after terraform apply for direct access.
15. Deployed with terraform init → validate → plan → apply and verified: Apache web pages load over HTTP, and private cross-region pinging (via VPC Peering) works successfully.

Result: Two EC2 instances running Apache in different AWS regions, privately connected via cross-region VPC Peering — fully provisioned and reproducible with a single terraform apply.